AI-ASSISTED CODING

ASSIGNMENT 16.3

NAME:V.SIDDARTHA

HALL:2303A52062

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- **Tables: Doctors, Patients, Appointments.**
- **Use AI to define constraints (unique IDs, valid dates).**
- **Generate queries:**
- **List all appointments for a specific doctor.**
- **Retrieve patient history by patient ID.**
- **Count total patients treated by each doctor.**

Expected Output:

- **Normalized schema and SQL queries with joins.**

CODE

```
PRAGMA foreign_keys = ON;


-- ======================================================================
-- Hospital Management System (SQLite3)
-- Normalized schema with constraints and required join queries.
-- Tables: Doctors, Patients, Appointments
-- ======================================================================


DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Patients;
DROP TABLE IF EXISTS Doctors;
```

```
CREATE TABLE Doctors (
   doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
   doctor_code TEXT NOT NULL UNIQUE,
   doctor_name TEXT NOT NULL,
   specialization TEXT NOT NULL,
   email TEXT NOT NULL UNIQUE,
   phone TEXT NOT NULL UNIQUE CHECK (length(phone) >= 10),
   created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Patients (
   patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
   patient_code TEXT NOT NULL UNIQUE,
   patient_name TEXT NOT NULL,
   date_of_birth TEXT NOT NULL
      CHECK (date(date_of_birth) IS NOT NULL),
   gender TEXT NOT NULL CHECK (gender IN ('M', 'F', 'Other')),
   email TEXT UNIQUE,
   phone TEXT UNIQUE,
   created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER trg_patients_dob_not_future_insert
BEFORE INSERT ON Patients
FOR EACH ROW
WHEN date(NEW.date_of_birth) > date('now')
BEGIN
```

```
    SELECT RAISE(ABORT, 'date_of_birth cannot be in the future');
END;

CREATE TRIGGER trg_patients_dob_not_future_update
BEFORE UPDATE OF date_of_birth ON Patients
FOR EACH ROW
WHEN date(NEW.date_of_birth) > date('now')
BEGIN
    SELECT RAISE(ABORT, 'date_of_birth cannot be in the future');
END;

CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date TEXT NOT NULL CHECK (date(appointment_date) IS NOT NULL),
    appointment_time TEXT NOT NULL CHECK (time(appointment_time) IS NOT NULL),
    diagnosis TEXT,
    notes TEXT,
    status TEXT NOT NULL DEFAULT 'Scheduled'
        CHECK (status IN ('Scheduled', 'Completed', 'Cancelled')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
    UNIQUE (doctor_id, appointment_date, appointment_time),
    UNIQUE (patient_id, appointment_date, appointment_time)
);
```

```
CREATE INDEX idx_appointments_doctor_id ON Appointments(doctor_id);
CREATE INDEX idx_appointments_patient_id ON Appointments(patient_id);
CREATE INDEX idx_appointments_date ON Appointments(appointment_date);

-- ----------------------------------------------------------------------
-- Sample data
-- ----------------------------------------------------------------------
INSERT INTO Doctors (doctor_code, doctor_name, specialization, email, phone) VALUES
('DOC001', 'Dr. Rajesh Kumar', 'Cardiology', 'rajesh.kumar@hospital.com', '9876543210'),
('DOC002', 'Dr. Priya Sharma', 'Neurology', 'priya.sharma@hospital.com', '9876543211'),
('DOC003', 'Dr. Amit Patel', 'Orthopedics', 'amit.patel@hospital.com', '9876543212');

INSERT INTO Patients (patient_code, patient_name, date_of_birth, gender, email, phone)
VALUES
('PAT001', 'Rahul Verma', '1985-05-15', 'M', 'rahul.verma@email.com', '9123456789'),
('PAT002', 'Anjali Singh', '1990-08-22', 'F', 'anjali.singh@email.com', '9123456790'),
('PAT003', 'Suresh Reddy', '1980-07-08', 'M', 'suresh.reddy@email.com', '9123456793'),
('PAT004', 'Divya Nair', '1995-11-25', 'F', 'divya.nair@email.com', '9123456792');

INSERT INTO Appointments (
    doctor_id,
    patient_id,
    appointment_date,
    appointment_time,
    diagnosis,
    notes,
    status
```

```
) VALUES
(1, 1, '2026-04-05', '09:30:00', 'Hypertension', 'BP readings normal after medication', 'Completed'),
(1, 3, '2026-04-13', '10:30:00', 'Cardiac evaluation', 'Follow-up for chest pain', 'Scheduled'),
(2, 2, '2026-04-07', '14:30:00', 'Migraine', 'Treatment effective; reduced frequency', 'Completed'),
(3, 4, '2026-04-15', '11:00:00', 'Back pain', 'Recommended physiotherapy', 'Scheduled'),
(1, 1, '2026-04-20', '09:00:00', 'Hypertension follow-up', 'Monitor blood pressure for 2 weeks', 'Scheduled');


-- ======================================================================
-- REQUIRED QUERY 1: List all appointments for a specific doctor
-- Parameter: :doctor_id
-- ======================================================================
SELECT
   d.doctor_id,
   d.doctor_name,
   d.specialization,
   a.appointment_id,
   a.appointment_date,
   a.appointment_time,
   p.patient_id,
   p.patient_name,
   a.diagnosis,
   a.status
FROM Appointments AS a
JOIN Doctors AS d ON d.doctor_id = a.doctor_id
JOIN Patients AS p ON p.patient_id = a.patient_id
WHERE a.doctor_id = :doctor_id
```

```
ORDER BY a.appointment_date, a.appointment_time;


-- ======================================================================
-- REQUIRED QUERY 2: Retrieve patient history by patient ID
-- Parameter: :patient_id
-- ======================================================================
SELECT
    p.patient_id,
    p.patient_name,
    p.date_of_birth,
    p.gender,
    d.doctor_id,
    d.doctor_name,
    d.specialization,
    a.appointment_id,
    a.appointment_date,
    a.appointment_time,
    a.diagnosis,
    a.notes,
    a.status
FROM Patients AS p
LEFT JOIN Appointments AS a ON a.patient_id = p.patient_id
LEFT JOIN Doctors AS d ON d.doctor_id = a.doctor_id
WHERE p.patient_id = :patient_id
ORDER BY a.appointment_date DESC, a.appointment_time DESC;


-- ======================================================================
-- REQUIRED QUERY 3: Count total patients treated by each doctor
```

-- "Treated" counted from completed appointments.

-- ==========

SELECT

d.doctor_id,

d.doctor_name,

d.specialization,

COUNT(DISTINCT CASE WHEN a.status = 'Completed' THEN a.patient_id END) AS total_patients_treated

FROM Doctors AS d

LEFT JOIN Appointments AS a ON a.doctor_id = d.doctor_id

GROUP BY d.doctor_id, d.doctor_name, d.specialization

ORDER BY total_patients_treated DESC, d.doctor_name;

OUTPUT

```
me | patient_id | patient_name | diagnosis | status
-----------------------------------------------------------------------------------------
-----------------------------------------------------
1 | Dr. Rajesh Kumar | Cardiology | 1 | 2026-04-05 | 09:30:00 | 1 | Rahul Verma | Hypertensio
n | Completed
1 | Dr. Rajesh Kumar | Cardiology | 2 | 2026-04-13 | 10:30:00 | 3 | Suresh Reddy | Cardiac ev
aluation | Scheduled
1 | Dr. Rajesh Kumar | Cardiology | 5 | 2026-04-20 | 09:00:00 | 1 | Rahul Verma | Hypertensio
n follow-up | Scheduled

Query 2: Patient History for Patient ID 1
----------------------------------------
patient_id | patient_name | date_of_birth | gender | doctor_id | doctor_name | specialization
 | appointment_id | appointment_date | appointment_time | diagnosis | notes | status
-----------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------
1 | Rahul Verma | 1985-05-15 | M | 1 | Dr. Rajesh Kumar | Cardiology | 5 | 2026-04-20 | 09:00
:00 | Hypertension follow-up | Monitor blood pressure for 2 weeks | Scheduled
1 | Rahul Verma | 1985-05-15 | M | 1 | Dr. Rajesh Kumar | Cardiology | 1 | 2026-04-05 | 09:30
:00 | Hypertension | BP readings normal after medication | Completed

Query 3: Total Patients Treated by Each Doctor
----------------------------------------------
doctor_id | doctor_name | specialization | total_patients_treated
-------------------------------------------------------------------
2 | Dr. Priya Sharma | Neurology | 1
1 | Dr. Rajesh Kumar | Cardiology | 1
3 | Dr. Amit Patel | Orthopedics | 0
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

powershell
powershell
Python
Python
Python
Python

1 Hidden Tern

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

- **Tables: Users, Products, Orders, OrderDetails.**
- **Generate queries to:**
- **Retrieve all orders by a user.**
- **Find the most purchased product.**
- **Calculate total revenue in a given month.**
- **AI should also suggest normalization improvements and query optimization.**

Expected Output:

- **Complete schema with relationships + SQL queries for analytics.**

CODE

```
PRAGMA foreign_keys = ON;


-- ======================================================================
-- Task 2: Online Shopping Database (SQLite3)
-- Tables: Users, Products, Orders, OrderDetails
-- Includes normalization and query optimization guidance.
-- ======================================================================


DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;


CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    user_code TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT UNIQUE,
    city TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    sku TEXT NOT NULL UNIQUE,
    product_name TEXT NOT NULL,
    category TEXT NOT NULL,
    unit_price REAL NOT NULL CHECK (unit_price >= 0),
    is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    order_date TEXT NOT NULL CHECK (date(order_date) IS NOT NULL),
    order_status TEXT NOT NULL DEFAULT 'PLACED'
        CHECK (order_status IN ('PLACED', 'SHIPPED', 'DELIVERED', 'CANCELLED')),
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON UPDATE CASCADE ON DELETE
RESTRICT
);
```

```
CREATE TABLE OrderDetails (
   order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
   order_id INTEGER NOT NULL,
   product_id INTEGER NOT NULL,
   quantity INTEGER NOT NULL CHECK (quantity > 0),
   unit_price_at_purchase REAL NOT NULL CHECK (unit_price_at_purchase >= 0),
   FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON UPDATE CASCADE ON DELETE
CASCADE,
   FOREIGN KEY (product_id) REFERENCES Products(product_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
   UNIQUE (order_id, product_id)
);

-- Indexing for common analytical filters and joins
CREATE INDEX idx_orders_user_date ON Orders(user_id, order_date);
CREATE INDEX idx_orderdetails_order_product ON OrderDetails(order_id, product_id);
CREATE INDEX idx_orderdetails_product ON OrderDetails(product_id);

-- Sample data
INSERT INTO Users (user_code, full_name, email, phone, city) VALUES
('USR001', 'Asha Reddy', 'asha@example.com', '9000000001', 'Hyderabad'),
('USR002', 'Ravi Kumar', 'ravi@example.com', '9000000002', 'Warangal'),
('USR003', 'Neha Sharma', 'neha@example.com', '9000000003', 'Nizamabad');

INSERT INTO Products (sku, product_name, category, unit_price) VALUES
('SKU1001', 'Wireless Mouse', 'Electronics', 799.00),
('SKU1002', 'Laptop Stand', 'Electronics', 1299.00),
('SKU1003', 'Water Bottle', 'Home', 499.00),
('SKU1004', 'Notebook Pack', 'Stationery', 299.00);
```

```
INSERT INTO Orders (user_id, order_date, order_status) VALUES
(1, '2026-03-10', 'DELIVERED'),
(1, '2026-03-21', 'DELIVERED'),
(2, '2026-03-25', 'SHIPPED'),
(3, '2026-04-02', 'PLACED'),
(1, '2026-04-10', 'DELIVERED');

INSERT INTO OrderDetails (order_id, product_id, quantity, unit_price_at_purchase)
VALUES
(1, 1, 2, 799.00),
(1, 3, 1, 499.00),
(2, 2, 1, 1299.00),
(2, 4, 3, 299.00),
(3, 1, 1, 799.00),
(3, 2, 1, 1299.00),
(4, 3, 2, 499.00),
(5, 1, 1, 799.00),
(5, 4, 2, 299.00);

-- ------------------------------------------------------------------
-- Query 1: Retrieve all orders by a user (replace 1 with desired user_id)
-- ------------------------------------------------------------------
SELECT
    u.user_id,
    u.full_name,
    o.order_id,
    o.order_date,
```

```
    o.order_status,
    SUM(od.quantity * od.unit_price_at_purchase) AS order_total
FROM Users AS u
JOIN Orders AS o ON o.user_id = u.user_id
JOIN OrderDetails AS od ON od.order_id = o.order_id
WHERE u.user_id = 1
GROUP BY u.user_id, u.full_name, o.order_id, o.order_date, o.order_status
ORDER BY o.order_date DESC;

-- ---------------------------------------------------------------------
-- Query 2: Find the most purchased product by total quantity
-- ---------------------------------------------------------------------
SELECT
    p.product_id,
    p.product_name,
    p.category,
    SUM(od.quantity) AS total_units_sold
FROM Products AS p
JOIN OrderDetails AS od ON od.product_id = p.product_id
JOIN Orders AS o ON o.order_id = od.order_id
WHERE o.order_status <> 'CANCELLED'
GROUP BY p.product_id, p.product_name, p.category
ORDER BY total_units_sold DESC, p.product_name
LIMIT 1;

-- ---------------------------------------------------------------------
-- Query 3: Calculate total revenue for a given month (example: 2026-03)
-- ---------------------------------------------------------------------
```

```
SELECT
   strftime('%Y-%m', o.order_date) AS revenue_month,
   ROUND(SUM(od.quantity * od.unit_price_at_purchase), 2) AS total_revenue
FROM Orders AS o
JOIN OrderDetails AS od ON od.order_id = o.order_id
WHERE o.order_status IN ('SHIPPED', 'DELIVERED')
  AND strftime('%Y-%m', o.order_date) = '2026-03'
GROUP BY strftime('%Y-%m', o.order_date);


-- ---------------------------------------------------------------------
-- AI suggestions: Normalization improvements
-- ---------------------------------------------------------------------
-- 1) Move product category text into Category table and reference category_id.
-- 2) Add UserAddresses table for multiple addresses per user.
-- 3) Add Payments table (one-to-many from Orders) for payment events.
-- 4) Add InventoryTransactions table to track stock changes over time.


-- ---------------------------------------------------------------------
-- AI suggestions: Query optimization
-- ---------------------------------------------------------------------
-- 1) Keep composite index on Orders(user_id, order_date) for user history queries.
-- 2) Keep index on OrderDetails(product_id) for top-product analytics.
-- 3) Use order_status filters to avoid counting cancelled orders in revenue.
-- 4) For very large data, consider pre-aggregated monthly sales summary tables.
```

OUTPUT

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- **Tables: Books, Members, Loans.**
- **Use AI to suggest indexing strategy for faster queries.**
- **Generate queries:**
- **Retrieve all books currently issued.**
- **Find overdue books (loan date > 30 days).**
- **Count number of books loaned by each member.**

Expected Output:

- **Schema + SQL queries demonstrating joins and conditions.**

CODE

```
PRAGMA foreign_keys = ON;


-- ======================================================================
-- Task 3: Library Management System (SQLite3)
-- Tables: Books, Members, Loans
-- Includes indexing strategy for faster queries.
-- ======================================================================


DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Members;
DROP TABLE IF EXISTS Books;


CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    isbn TEXT NOT NULL UNIQUE,
    title TEXT NOT NULL,
```

```
    author_name TEXT NOT NULL,
    published_year INTEGER CHECK (published_year BETWEEN 1500 AND 2100),
    total_copies INTEGER NOT NULL CHECK (total_copies >= 1),
    available_copies INTEGER NOT NULL CHECK (available_copies >= 0 AND available_copies
<= total_copies)
);

CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    member_code TEXT NOT NULL UNIQUE,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    joined_on TEXT NOT NULL CHECK (date(joined_on) IS NOT NULL)
);

CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date TEXT NOT NULL CHECK (date(loan_date) IS NOT NULL),
    due_date TEXT NOT NULL CHECK (date(due_date) IS NOT NULL),
    return_date TEXT,
    FOREIGN KEY (book_id) REFERENCES Books(book_id) ON UPDATE CASCADE ON DELETE
RESTRICT,
    FOREIGN KEY (member_id) REFERENCES Members(member_id) ON UPDATE CASCADE
ON DELETE RESTRICT,
    CHECK (return_date IS NULL OR date(return_date) IS NOT NULL),
    CHECK (date(due_date) >= date(loan_date))
);
```

```
-- Indexing strategy
CREATE INDEX idx_loans_active ON Loans(return_date, loan_date);
CREATE INDEX idx_loans_book ON Loans(book_id);
CREATE INDEX idx_loans_member ON Loans(member_id);
CREATE INDEX idx_loans_due_date ON Loans(due_date);

-- Sample data
INSERT INTO Books (isbn, title, author_name, published_year, total_copies,
available_copies) VALUES
('9780132350884', 'Clean Code', 'Robert C. Martin', 2008, 4, 2),
('9780134685991', 'Effective Java', 'Joshua Bloch', 2018, 3, 1),
('9781491957660', 'Designing Data-Intensive Applications', 'Martin Kleppmann', 2017, 2,
0),
('9780201633610', 'Design Patterns', 'Erich Gamma', 1994, 5, 3);

INSERT INTO Members (member_code, full_name, email, joined_on) VALUES
('MEM001', 'Anita Rao', 'anita@example.com', '2025-01-10'),
('MEM002', 'Karthik R', 'karthik@example.com', '2025-02-18'),
('MEM003', 'Meera Das', 'meera@example.com', '2025-05-05');

INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES
(1, 1, '2026-03-01', '2026-03-31', NULL),
(2, 2, '2026-02-01', '2026-03-01', NULL),
(3, 1, '2026-01-15', '2026-02-14', '2026-02-10'),
(3, 3, '2026-03-10', '2026-04-09', NULL),
(4, 2, '2026-03-20', '2026-04-19', NULL);

-- ----------------------------------------------------------------------
```

```
-- Query 1: Retrieve all books currently issued (not yet returned)
-- ----------------------------------------------------------------------
SELECT
   l.loan_id,
   b.book_id,
   b.title,
   b.author_name,
   m.member_id,
   m.full_name,
   l.loan_date,
   l.due_date
FROM Loans AS l
JOIN Books AS b ON b.book_id = l.book_id
JOIN Members AS m ON m.member_id = l.member_id
WHERE l.return_date IS NULL
ORDER BY l.loan_date;


-- ----------------------------------------------------------------------
-- Query 2: Find overdue books (loaned for more than 30 days and not returned)
-- ----------------------------------------------------------------------
SELECT
   l.loan_id,
   b.title,
   m.full_name,
   l.loan_date,
   CAST(julianday('now') - julianday(l.loan_date) AS INTEGER) AS days_since_loan
FROM Loans AS l
JOIN Books AS b ON b.book_id = l.book_id
```

```
JOIN Members AS m ON m.member_id = l.member_id
WHERE l.return_date IS NULL
  AND julianday('now') - julianday(l.loan_date) > 30
ORDER BY days_since_loan DESC;


-- ---------------------------------------------------------------------
-- Query 3: Count number of books loaned by each member
-- ---------------------------------------------------------------------
SELECT
    m.member_id,
    m.full_name,
    COUNT(l.loan_id) AS books_loaned_count
FROM Members AS m
LEFT JOIN Loans AS l ON l.member_id = m.member_id
GROUP BY m.member_id, m.full_name
ORDER BY books_loaned_count DESC, m.full_name;
```

Task 4: Optimize Queries

Instructions:

- **Use AI tools to analyze query efficiency.**
- **Optimize inefficient queries using joins, indexes, or reducing subqueries.**
- **Test optimized queries for correctness.**

Starter Code Example:

```
-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
-- Optimized query
```

```
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Expected Output:

- **Both queries return the same result: all books written by UK authors.**
- **Optimized query performs faster for larger datasets.**

CODE

```
PRAGMA foreign_keys = ON;

-- ======================================================================
-- Task 4: Optimize Queries (SQLite3)
-- Goal: show original and optimized query return same results.
-- ======================================================================

DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Authors;

CREATE TABLE Authors (
   author_id INTEGER PRIMARY KEY AUTOINCREMENT,
   author_name TEXT NOT NULL,
   country TEXT NOT NULL
);

CREATE TABLE Books (
   book_id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT NOT NULL,
   author_id INTEGER NOT NULL,
```

```
    published_year INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(author_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Indexes for optimization
CREATE INDEX idx_authors_country ON Authors(country);
CREATE INDEX idx_books_author_id ON Books(author_id);

INSERT INTO Authors (author_name, country) VALUES
('Arthur Conan Doyle', 'UK'),
('J. K. Rowling', 'UK'),
('Mark Twain', 'USA'),
('Kazuo Ishiguro', 'UK'),
('Haruki Murakami', 'Japan');

INSERT INTO Books (title, author_id, published_year) VALUES
('Sherlock Holmes', 1, 1892),
('Harry Potter 1', 2, 1997),
('Harry Potter 2', 2, 1998),
('The Adventures of Tom Sawyer', 3, 1876),
('Never Let Me Go', 4, 2005),
('Kafka on the Shore', 5, 2002);

-- ---------------------------------------------------------------------
-- Original query (subquery)
-- ---------------------------------------------------------------------
SELECT *
```

```
FROM Books
WHERE author_id IN (
   SELECT author_id
   FROM Authors
   WHERE country = 'UK'
);

-- ---------------------------------------------------------------
-- Optimized query (join)
-- ---------------------------------------------------------------
SELECT b.*
FROM Books AS b
JOIN Authors AS a ON b.author_id = a.author_id
WHERE a.country = 'UK';

-- ---------------------------------------------------------------
-- Query plan analysis
-- ---------------------------------------------------------------
EXPLAIN QUERY PLAN
SELECT *
FROM Books
WHERE author_id IN (
   SELECT author_id
   FROM Authors
   WHERE country = 'UK'
);

EXPLAIN QUERY PLAN
```

```
SELECT b.*
FROM Books AS b
JOIN Authors AS a ON b.author_id = a.author_id
WHERE a.country = 'UK';


-- ---------------------------------------------------------------------
-- Correctness check: symmetric difference should return 0 rows
-- ---------------------------------------------------------------------
SELECT book_id, title, author_id, published_year FROM (
  SELECT *
  FROM Books
  WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country = 'UK'
  )
  EXCEPT
  SELECT b.*
  FROM Books AS b
  JOIN Authors AS a ON b.author_id = a.author_id
  WHERE a.country = 'UK'
)
UNION ALL
SELECT book_id, title, author_id, published_year FROM (
  SELECT b.*
  FROM Books AS b
  JOIN Authors AS a ON b.author_id = a.author_id
  WHERE a.country = 'UK'
```

```
  EXCEPT
  SELECT *
  FROM Books
  WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country = 'UK'
  )
);
```

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

- **Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.**
- **Define relationships:**
- **Each student can register for multiple courses.**
- **Each course is taught by a faculty member.**
- **Write SQL queries to:**
- **List all students enrolled in a specific course.**
- **Find faculty members teaching more than 2 courses.**
- **Retrieve courses with the highest number of registrations.**

CODE

```
PRAGMA foreign_keys = ON;


-- ======================================================================
```

```
-- Task 5: University Course Registration (SQLite3)
-- Tables: Students, Courses, Faculty, Registrations
-- ====================================================================

DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Students;
DROP TABLE IF EXISTS Faculty;

CREATE TABLE Faculty (
   faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
   faculty_code TEXT NOT NULL UNIQUE,
   faculty_name TEXT NOT NULL,
   department TEXT NOT NULL,
   email TEXT NOT NULL UNIQUE
);

CREATE TABLE Students (
   student_id INTEGER PRIMARY KEY AUTOINCREMENT,
   roll_no TEXT NOT NULL UNIQUE,
   student_name TEXT NOT NULL,
   email TEXT NOT NULL UNIQUE,
   semester INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 8)
);

CREATE TABLE Courses (
   course_id INTEGER PRIMARY KEY AUTOINCREMENT,
   course_code TEXT NOT NULL UNIQUE,
```

```
    course_title TEXT NOT NULL,
    credits INTEGER NOT NULL CHECK (credits BETWEEN 1 AND 6),
    faculty_id INTEGER NOT NULL,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    registration_date TEXT NOT NULL CHECK (date(registration_date) IS NOT NULL),
    grade TEXT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON UPDATE CASCADE ON DELETE RESTRICT,
    UNIQUE (student_id, course_id)
);

CREATE INDEX idx_courses_faculty_id ON Courses(faculty_id);
CREATE INDEX idx_registrations_course_id ON Registrations(course_id);
CREATE INDEX idx_registrations_student_id ON Registrations(student_id);

-- Sample data
INSERT INTO Faculty (faculty_code, faculty_name, department, email) VALUES
('FAC001', 'Dr. S. Prakash', 'CSE', 'prakash@sruni.edu'),
('FAC002', 'Dr. M. Devi', 'CSE', 'mdevi@sruni.edu'),
('FAC003', 'Dr. R. Iqbal', 'ECE', 'riqbal@sruni.edu');
```

```
INSERT INTO Students (roll_no, student_name, email, semester) VALUES
('2303A52001', 'Akhil Varma', 'akhil@sruni.edu', 4),
('2303A52002', 'Bhavya N', 'bhavya@sruni.edu', 4),
('2303A52003', 'Charan K', 'charan@sruni.edu', 6),
('2303A52004', 'Deepa S', 'deepa@sruni.edu', 2),
('2303A52005', 'Eshan R', 'eshan@sruni.edu', 6);

INSERT INTO Courses (course_code, course_title, credits, faculty_id) VALUES
('CSE201', 'Database Systems', 4, 1),
('CSE202', 'Operating Systems', 4, 1),
('CSE203', 'Computer Networks', 3, 2),
('CSE204', 'Machine Learning', 3, 2),
('ECE205', 'Digital Signal Processing', 4, 3),
('CSE205', 'Web Technologies', 3, 1);

INSERT INTO Registrations (student_id, course_id, registration_date, grade) VALUES
(1, 1, '2026-01-10', 'A'),
(1, 2, '2026-01-10', 'A-'),
(1, 3, '2026-01-10', 'B+'),
(2, 1, '2026-01-11', 'B'),
(2, 4, '2026-01-11', NULL),
(3, 2, '2026-01-12', 'A'),
(3, 3, '2026-01-12', 'A-'),
(3, 5, '2026-01-12', NULL),
(4, 1, '2026-01-13', NULL),
(4, 6, '2026-01-13', NULL),
(5, 1, '2026-01-13', 'A+'),
```

```
(5, 2, '2026-01-13', NULL),
(5, 6, '2026-01-13', NULL);

-- ----------------------------------------------------------------
-- Query 1: List all students enrolled in a specific course
-- Replace 'CSE201' with desired course code.
-- ----------------------------------------------------------------
SELECT
    c.course_code,
    c.course_title,
    s.student_id,
    s.roll_no,
    s.student_name,
    r.registration_date,
    r.grade
FROM Registrations AS r
JOIN Students AS s ON s.student_id = r.student_id
JOIN Courses AS c ON c.course_id = r.course_id
WHERE c.course_code = 'CSE201'
ORDER BY s.student_name;

-- ----------------------------------------------------------------
-- Query 2: Find faculty members teaching more than 2 courses
-- ----------------------------------------------------------------
SELECT
    f.faculty_id,
    f.faculty_name,
    f.department,
```

```
    COUNT(c.course_id) AS courses_taught
FROM Faculty AS f
JOIN Courses AS c ON c.faculty_id = f.faculty_id
GROUP BY f.faculty_id, f.faculty_name, f.department
HAVING COUNT(c.course_id) > 2
ORDER BY courses_taught DESC;


-- ----------------------------------------------------------------
-- Query 3: Retrieve courses with the highest number of registrations
-- ----------------------------------------------------------------
SELECT
    c.course_id,
    c.course_code,
    c.course_title,
    COUNT(r.registration_id) AS registration_count
FROM Courses AS c
LEFT JOIN Registrations AS r ON r.course_id = c.course_id
GROUP BY c.course_id, c.course_code, c.course_title
ORDER BY registration_count DESC, c.course_code;
```